

Attachment 1 — Architecture Overview

NLnet NGI Zero Commons Fund — Symphact application, 13th open call (deadline 2026-06-01).
Excerpt from `docs/vision-en.md` v1.4. Full text: [vision-en.md](#).

1. Design principles

Symphact is a capability-based actor runtime for .NET, co-designed with the Cognitive Fabric Processing Unit (CFPU). Eight invariants drive every design decision:

1. **Everything is an actor.** Device drivers, supervisors, services, business logic — all actors. No exception.
2. **No shared memory, ever.** Cores and actors communicate only via immutable messages through mailboxes.
3. **Let it crash.** A failing actor is restarted by its supervisor — no defensive error handling per actor.
4. **Supervision hierarchy.** Every actor has a supervisor; failures propagate up the tree until handled.
5. **Location transparency.** An actor reference does not reveal whether the target is local, remote, or on another chip.
6. **Capability-based security.** An actor can send a message to another only if it holds a capability — an unforgeable reference. No global namespace.
7. **Hot code loading.** A running system can accept new code without downtime (Erlang-style).
Planned, deferred to follow-up grant.
8. **Determinism by default (per actor).** Each actor sees its own messages in deterministic FIFO order — reproducible bugs, replay debugging, formal verification.

2. The three core primitives

The actor core is intentionally narrow — three primitives form the entire contract:

Primitive	File	Description
<code>IMailbox</code> / <code>TMailbox</code>	<code>src/Symphact.Core/IMailbox.cs</code> , <code>TMailbox.cs</code>	FIFO mailbox on <code>ConcurrentQueue<object></code> (lock-free MPMC). A future <code>TMmioMailbox</code> against CFPU hardware FIFO drops in without API changes.
<code>TActorRef</code>	<code>src/Symphact.Core/TActorRef.cs</code>	readonly record struct <code>TActorRef(int SlotIndex)</code> — opaque CST (Capability Slot Table) index. <code>default</code> is invalid (<code>TActorRef.Invalid</code>). Equality/hash by <code>SlotIndex</code> .
<code>TActor<TState></code> + <code>TActorSystem</code>	<code>src/Symphact.Core/TActor.cs</code> , <code>TActorSystem.cs</code>	<code>TActor<TState></code> is abstract (<code>Init()</code> , <code>Handle(state, msg)</code>); <code>TActorSystem</code> is the runtime (<code>Spawn</code> , <code>Send</code> , <code>DrainAsync</code> , <code>QuiesceAsync</code>).

Scheduling is decoupled via `IScheduler` (M0.4, `src/Symphact.Core/IScheduler.cs`):

- **TInlineScheduler** — synchronous, single-threaded reference; preserves strict global ordering.
- **TDedicatedThreadScheduler** — one OS thread per actor, simulating the CFPU dedicated-core-per-actor model (default cap 1000 actors, configurable).

Supervision (M0.3) provides `ISupervisorStrategy`, `OneForOne` / `AllForOne` strategies, lifecycle hooks (`PreStart`, `PostStop`, `PreRestart`, `PostRestart`), and the actor hierarchy.

Persistence (M0.5) provides `IJournal` / `ISnapshotStore`. The BCL-only in-memory reference implementations have shipped (44 tests); the content-addressed production-grade variant (`TCasJournal`, `TCasSnapshotStore`) is the central deliverable of grant milestone M1.

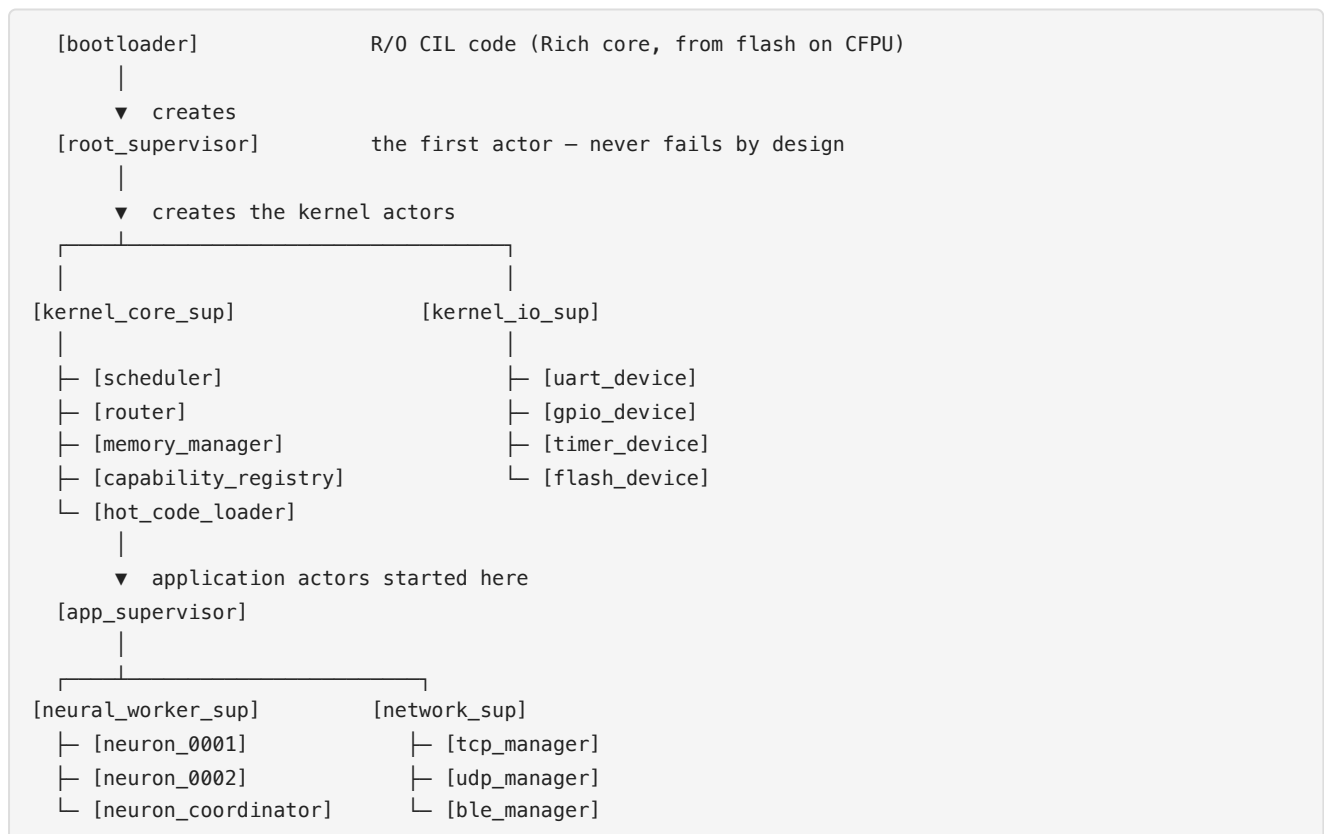
3. The unified primitive thesis

The same `TActor<TState>` primitive will serve four distinct roles on a single homogeneous core grid:

Role	Class	What it does
Classical OS process	<code>TCounterActor</code> , <code>TServiceActor</code> , ...	Plain stateful entity processing typed messages
Hardware driver	<code>TDeviceActor</code> → <code>uart_device</code> , <code>gpio_device</code> , <code>timer_device</code>	Owns an MMIO region; HW interrupt → mailbox message
AI agent	<code>TAgentActor</code> (LLM-driven)	Capability-restricted tool use, structured audit trail
Neuromorphic neuron	<code>TNeuronActor</code> (LIF / Izhikevich)	Membrane potential = state; <code>SpikeMsg(weight: int)</code> = message; <code>TActorRef</code> = synapse

Nobody builds this homogeneous integration openly: Loihi is SNN-only; Akka.NET is userland-only; seL4 is security-only; Linux is classical OS only. The unified primitive thesis is the central technical claim of this grant — *measured*, not just asserted, via a documented benchmark across the four reference actors on the same runtime.

4. Boot-time actor hierarchy



The kernel actors run on **a few Rich cores** but are **not in kernel mode** — Symphact has no kernel/user separation. Hardware isolation (shared-nothing multi-core) replaces what other OSes achieve with mode-switch overhead.

5. Message routing — four levels

A message reaches its target via one of four routes, in order of increasing cost:

Route	Latency (CFPU target)	Mechanism
Local (same core, same actor)	~1-3 cycles	Zero-copy, internal
Inter-core (same chip, different core)	~10-20 cycles	Hardware mailbox FIFO
Inter-chip (different Symphact node)	~100-1000 cycles	Dedicated network actor
Wide area (geographic)	~ms	Network actor over TCP

The router handles all four **transparently**. The developer always writes `Send(ref, msg)`.

Backpressure is **natural** — the hardware FIFO has a fixed depth (8 slots on F3, ~64 on F6), so a full mailbox blocks the sender (or returns `SendError` on `TrySend`). No explicit rate limiting needed.

6. Capability-based security — the CST model

Every `TActorRef` is an **opaque 32-bit CST (Capability Slot Table) index**:

```
public readonly record struct TActorRef(int SlotIndex);
// Capability data (perms, actor-id, core-coord) resides in the HW-managed
// CST — NOT in the token itself.
```

The CST slot is allocated by `capability_registry` (M2.5). On every incoming message, the **target-core mailbox-edge HW unit** performs a CST lookup — NOT the interconnect router. This is consistent with the CFPU "one mailbox IRQ per core" principle.

On an invalid CST slot, the HW unit:

1. **Drops** the message,
2. Triggers a **fail-stop** on the sender core,
3. Initiates **AuthCode quarantine** against the signer (LMS hash-based signature blacklist).

On .NET hosts the same model is enforced by the runtime (`TCapabilityRegistry`); the HW path is engaged only on CFPU silicon.

Delegation: an actor passes its `TActorRef` in a message — the receiver gets the capability (permissions may be attenuated). **Revocation**: the issuer invalidates the CST slot; all clones become invalid simultaneously.

Isolation guarantees (on CFPU):

- An actor **cannot** write to another actor's memory — hardware physically prevents it.
- An actor **cannot** call another actor's code — only messages.
- An actor **cannot** access a peripheral unless it holds the device actor's reference.
- An actor **cannot** spoof messages — the target-core CST lookup verifies the sender's capability.

7. Per-core memory model

Resource	Scope
Eval stack, local variables, frames	Per-core SRAM (private, 16-256 KB depending on core type)
Heap	Rich cores only (Nano cores have no heap allocator)
GC	Per-core bump allocator + mark-sweep; no global stop-the-world
Inter-core data	Message copying through the mailbox FIFO (~10-30 cycles, depending on size)

In Akka.NET the global .NET GC is the bottleneck under high load; on Symphact + CFPU **this problem does not exist** by construction.

8. .NET independence

The CIL specification (ECMA-335) is ratified as ISO/IEC 23271. Symphact targets the bytecode format, not any specific Microsoft runtime. Alternative CIL implementations exist (Mono, legacy .NET Framework compilers, Roslyn-independent front-ends). The runtime design operates at CIL level and is independent of upstream runtime changes.

Related documents:

- [docs/vision-en.md](#) — full vision (v1.4)
- [docs/roadmap-en.md](#) — phased implementation plan (v1.0)
- [docs/trust-model-en.md](#) — trust chain and capability model details
- [CLAUDE.md](#) — repository-level invariants and conventions